

Homework 5: Disciplined Convex Programming and Data Fitting

Instructions

Please submit a .qmd file along with a rendered pdf to the Brightspace page for this assignment. You may use whatever language you like within your qmd file, I recommend python, julia, or R.

Problem 1: Penalty Function Approximations (Modified from Exercise 4 in CVX Book Extended Exercises)

Consider the approximation problem:

$$\min_{\mathbf{x} \in \mathbb{R}^n} \phi(A\mathbf{x} - \mathbf{b}),$$

where A is an $m \times n$ matrix, $x \in \mathbb{R}^n$, and $\phi : \mathbb{R}^m \rightarrow \mathbb{R}$ is a convex penalty function measuring the approximation error, and $\mathbf{b}$ is an m -vector.

The purpose of this exercise is for you to implement several different penalty functions in CVX and study how the resulting coefficients x from each penalty function differ, as a means of building intuition about penalty functions.

You will use the following penalty functions:

- (a) $\phi(\mathbf{y}) = \|\mathbf{y}\|_2$, the standard Euclidean norm
- (b) $\phi(\mathbf{y}) = \|\mathbf{y}\|_1$, the L_1 norm. This is often referred to as the Lasso
- (c) $\phi(\mathbf{y}) = \sum_{k=1}^{m/2} |y_{r_k}|$, where r_k is the index of the component with the k th largest absolute value. This is like the Lasso, but where we only count the terms with their error in the top half, i.e. y_{r_1} is the y with largest absolute value, y_{r_2} is the y with second largest absolute value, etc.
- (d) $\phi(\mathbf{y}) = \sum_{k=1}^m h(y_k)$, where $h(y)$ is the Huber penalty, defined by:

$$h(u) = \begin{cases} u^2, & |u| \leq M \\ M(2|u| - M), & |u| \geq M, \end{cases}$$

For this problem use $M = 0.2$

(e) $\phi(\mathbf{y}) = \sum_{k=1}^m h(y_k)$, where h is the log-barrier penalty, defined by:

$$h(u) = -\log(1 - u^2), \quad \text{dom}(h) = \{u \mid |u| < 1\}$$

Generate data A and $\mathbf{b}$ as follows:

- $m = 200$
- $n = 100$
- $A_{ij} \sim \text{Normal}(\mu = 0, \sigma = 1)$, each element normally distributed with mean 0 and standard deviation 1
- Initialize b as using a normal distribution of mean $\mu = 0$ and $\sigma = 1$, and then normalize b so that all of its entries have absolute value less than 1 by doing something like:
 - $b_i \sim \text{Normal}(\mu = 0, \sigma = 1)$
 - and then: $\mathbf{b} = \mathbf{b} / (1.01 \cdot \max(\text{abs}(\mathbf{b})))$

This is to make sure the **log-barrier** function as a non-empty domain.

Visualize the distribution of errors (using a tool like a histogram or density plot) for each of these penalty function formulations and comment on the differences that you observe. Each penalty function prioritizes a errors differently, how do these priorities manifest in the distribution of residuals.

Some hints for selected parts:

- (a) Technically this is a least squares problem, you can solve it using Least-Squares formula or **CVX**
- (b) Use **norm(y,1)**
- (c) Use **norm_largest()**
- (d) Use **huber()**
- (e) The extended exercises claimed that the **log-barrier** objective needed to be reformulated to use the geometric mean, but I found that this problem worked perfectly well with a straightforward implementation. I suspect that the **CVX** software was upgraded to better handle **log** and **exp** objecties since this exercise was developed.

Problem 2: Fitting Censored Data (Extended Exercises 6.13 in CVX Book)

In some experiments there are two kinds of measurements or data available: The usual ones, in which you get a number (say), and censored data, in which you don't get the specific number, but are told something about it, such as a lower bound. A classic example is a study of lifetimes of a set of subjects (say, laboratory mice, devices undergoing reliability testing, or people in a long-term, longitudinal study). For those who have died by the end of data collection, we get the lifetime. For those who have not died by the end of data collection, we do not have the lifetime, but we do have a lower bound, i.e., the length of the study. In statistics, we call this type of data **right-censored** data, meaning that we do not have the exact values in the right tail of the distribution. The data points that are not present are called the censored data values.

We wish to fit a set of data points, $((\mathbf{x}_1, y_1), \dots, (\mathbf{x}_k, y_k))$, with $\mathbf{x}_k \in \mathbb{R}^n$ and $y_k \in \mathbb{R}$, with a linear model of the form $y \approx \mathbf{c}^T \mathbf{x}$. The vector $\mathbf{c} \in \mathbb{R}^n$ is the model parameter, which we want to choose. We will use a least-squares criterion, i.e., choose $\mathbf{c}$ to minimize:

$$J = \sum_{i=1}^k (y_i - \mathbf{c}^T \mathbf{x}_i)^2$$

Here is the tricky part: some of the values of y_i are censored; for these entries, we have only a (given) lower bound. We will re-order the data so that $y_1, \dots, y_m$ are given (i.e., uncensored), while $y_{m+1}, \dots, y_k$ are all censored, i.e., unknown, but larger than D , a given number. All the values of $\mathbf{x}_i$ are known.

- (a) Explain how to find $\mathbf{c}$ (the model parameter) and $y_{m+1}, \dots, y_k$ (the censored data values) that minimize J . Hint: should the censored data be variables or parameters?
- (b) Carry out the method of part (a) on the data values in the file [censored_dict.json](#). You can process this file in R using `fromJSON` in the `jsonlite` package or in python using the `json` library and:

```
with open('censored_dict.json', 'r') as fp:
    data = json.load(fp)
```

Report $\hat{\mathbf{c}}$, the value of $\mathbf{c}$ found using this method. Also find $\hat{\mathbf{c}}_{ls}$, the least-squares estimate of $\mathbf{c}$ obtained by simply ignoring the censored data samples, i.e., the least-squares estimate based on the data $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)$. The data file contains $\mathbf{c}_{\text{true}}$, the true value of $\mathbf{c}$, in the vector `$\mathbf{c}_{\text{true}}$` . Use this to give the two relative errors:

$$\frac{\|\mathbf{c}_{\text{true}} - \hat{\mathbf{c}}\|_2^2}{\|\mathbf{c}_{\text{true}}\|_2^2}, \quad \frac{\|\mathbf{c}_{\text{true}} - \hat{\mathbf{c}}_{ls}\|_2^2}{\|\mathbf{c}_{\text{true}}\|_2^2}$$

Problem 3: Robust Logistic Regression (Exercise 6.29 in the CVX Book extended exercises)

We are given a data set $\mathbf{x}_i \in \mathbb{R}^d$, $y_i \in \{-1, 1\}$, $i = 1, \dots, n$. We seek a prediction model $\hat{y} = \text{sign}(\theta^T \mathbf{x})$, where $\theta \in \mathbb{R}^d$ is the model parameter. In logistic regression, θ is chosen as the minimizer of the logistic loss:

$$l(\theta) = \sum_{i=1}^n \log(1 + \exp(-y_i \theta^T \mathbf{x}_i))$$

which is a convex function of θ . Here $\|\delta_i\|_\infty = \max_j |\delta_{ij}|$. Remember that each δ_i is a vector with length the same as $\mathbf{x}_i$.

In robust regression, we take into account the idea that the feature vectors $\mathbf{x}_i$ are not known precisely. Specifically we imagine that each entry of each feature vector can vary by $\pm\epsilon$, where $\epsilon > 0$ is a given uncertainty level. We define the worst-case logistic loss as:

$$l_{wc}(\theta) = \sum_{i=1}^n \sup_{\|\delta_i\|_\infty \leq \epsilon} \log(1 + \exp(-y_i \theta^T (\mathbf{x}_i + \delta_i)))$$

In words: we perturb each feature vector's entries by up to ϵ in such a way as to make the logistic loss as large as possible. Each term is convex, since it is the supremum of a family of convex functions of θ , and so $l_{wc}(\theta)$ is a convex function of θ .

In robust logistic regression, we choose θ to minimize $l_{wc}(\theta)$.

- Explain how to carry out robust logistic regression by solving a single convex optimization problem in disciplined convex programming (DCP) form. Justify any change of variables or introduction of new variables. Explain why solving the problem you propose also solves the robust logistic regression problem. Hint: $\log(1 + \exp(u))$ is monotonic in u .
- Fit a standard logistic regression model (i.e., minimize $l(\theta)$), and also a robust logistic regression model (i.e., minimize $l_{wc}(\theta)$), using the data given in [rob_regression.csv](#) and [rob_regression_test.csv](#). The $\mathbf{x}_i$ s are provided as the rows of an $n \times d$ matrix named X (these are the variables of the data frame named "X_1, X_2, ..."). The y_i s are provided as the entries of a n -vector named y (the first column in the data frame). The file also contains a test data set, $X_{\text{test}}, y_{\text{test}}$. Give the test error rate (i.e., fraction of test set data points for which $\hat{y} = y$) for the logistic regression and robust logistic regression models.